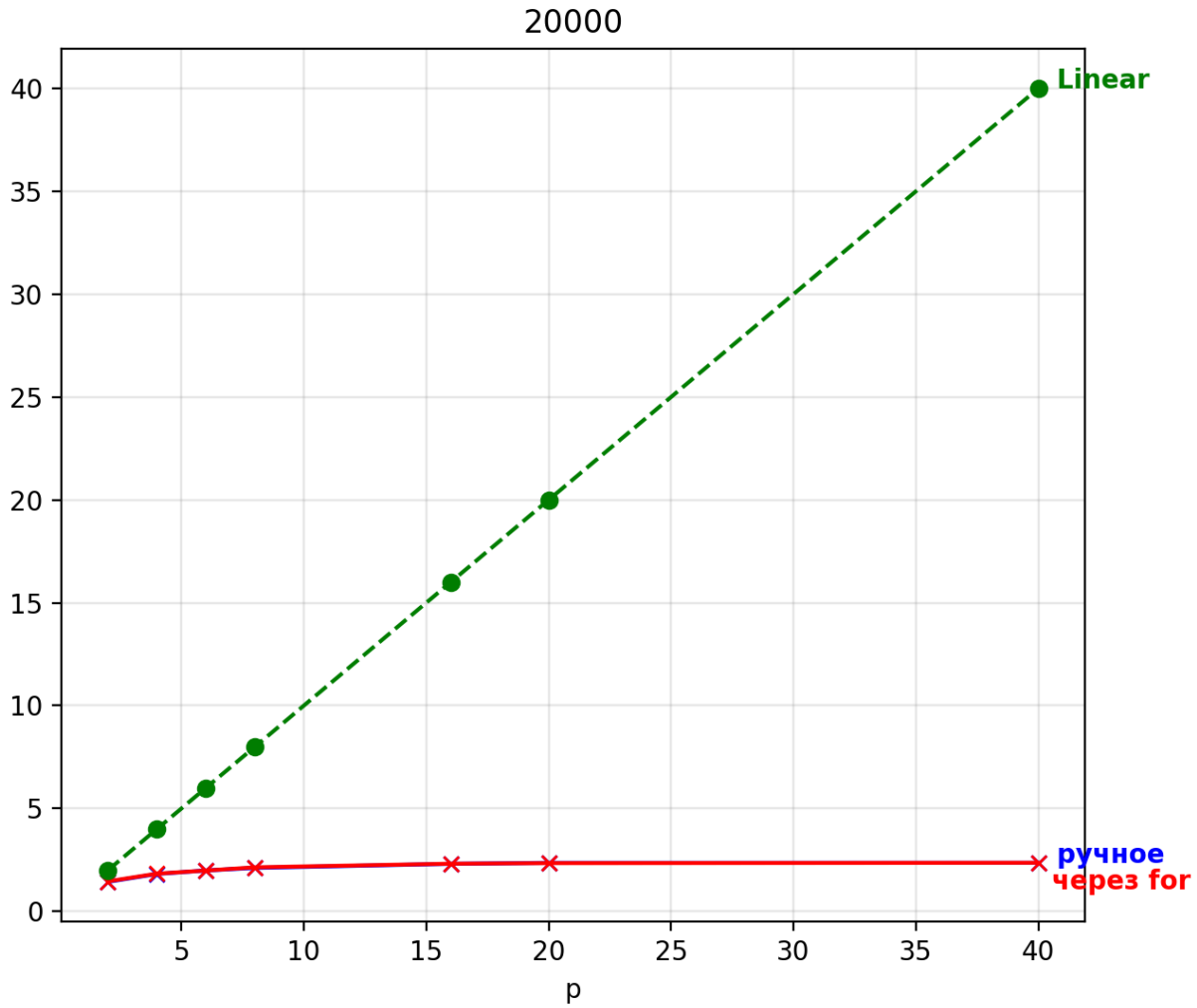
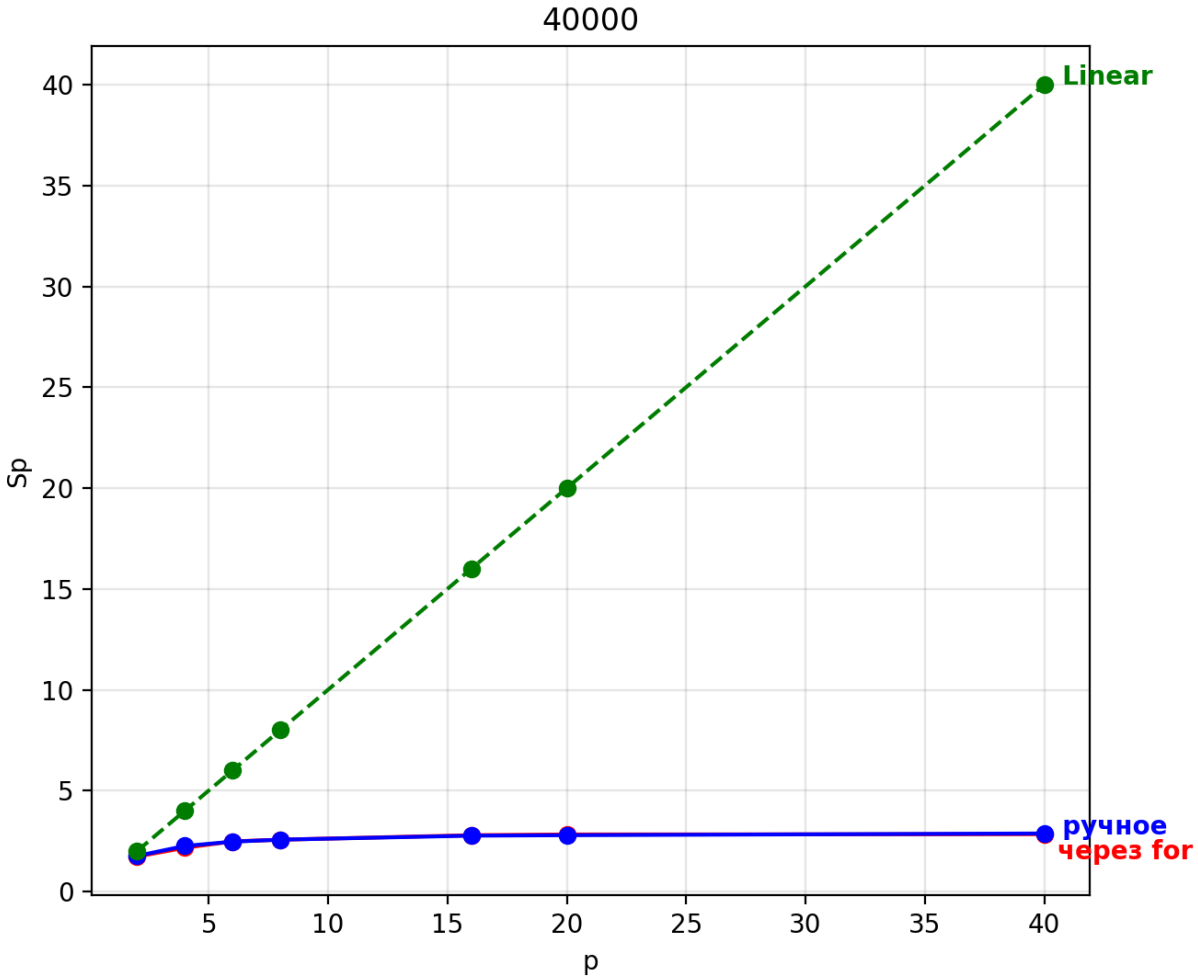
Задание 1

M × N	Количество потоков														
	2			4		6		8		16		20		40	
	T_1	T_2	S_2	T_4	S_4	T_6	S_6	T_8	S_8	T_{16}	S_{16}	T_{20}	S_{20}	T_{40}	S_{40}
20000 × 20000 ручное	4.70138	3.2695	1.4379	2.5840	1.8194	2.3641	1.9887	2.2237	2.1142	2.0331	2.3124	1.9911	2.3612	1.9923	2.3598
20000 × 20000 через for	4.70138	3.2193	1.4604	2.5629	1.8344	2.3660	1.9871	2.1954	2.1414	2.0282	2.3180	2.0036	2.3465	1.9885	2.3643
40000 × 40000 ручное	22.8911	13.2178	1.7318	10.5665	2.1664	9.2231	2.4819	8.9210	2.5660	8.2077	2.7890	8.1000	2.8260	8.0603	2.8400
40000 × 40000 через for	22.8911	12.9944	1.7616	10.1473	2.2559	9.2692	2.4696	8.9117	2.5686	8.2754	2.7661	8.2058	2.7896	7.9610	2.8754





В ходе работы были реализованы последовательный и параллельный алгоритмы умножения матрицы на вектор.
Наблюдается сильно затухающий рост ускорения при увеличении числа потоков. При N=20000 и N=40000 ускорение оказалось почти одинаковым, поэтому главным ограничивающим фактором является пропускная способность памяти.
Идеальное ускорение не было достигнуто по следующим причинам:

- Время на инициализацию потоков.
- Пропускная способность оперативной памяти стала узким местом при большом количестве потоков, обращающихся к памяти одновременно.

Параллельная реализация умножения матрицы на вектор демонстрирует плохую масштабируемость. Основным фактором, ограничивающим линейное ускорение, является пропускная способность оперативной памяти, Для дальнейшего повышения производительности необходима оптимизация работы с памятью и кэш-ориентированные алгоритмы.